Solution: Capacity To Ship Packages Within D Days

Problem Statement

You are given `weights` array of size `n`, where `weights[i]` represents the weight of the i^{th} package. These `n` packages should be shipped from one place to another place via one ship within a given `days`.

If you can load the ship with some packages each day (in the order given in `weights[i]`), find the smallest possible maximum weight the ship can carry per day without exceeding the given number of days for shipping all packages.

Examples

Example 1:

- **Input**: weights = `[10,20,30,40,50]`, days = `3`
- **Expected Output**: `60`
- **Justification**: A capacity of 60 means shipping [10,20,30] on day 1, [40] on day 2, and [50] on day 3.

Example 2:

- **Input**: weights = `[5,5,5,5,5,5,5,5,5,5]`, days = `5`
- **Expected Output**: `10`
- **Justification**: A capacity of 10 allows shipping two packages of weight 5 each day, perfectly fitting within 5 days.

Example 3:

- **Input**: weights = `[4, 8, 5, 2, 6, 3, 7, 9]`, days = `4`
- **Expected Output**: `13`
- **Justification**: A capacity of 13 allows shipping [4, 8] on day 1 (total 12, close to 13 without exceeding), [5, 2, 6] on day 2 (total 13), [3, 7] on day 3 (total 10), and [9] on day 4.

Solution

To solve this problem, we'll use a binary search approach. This method is effective because it efficiently narrows down the minimum capacity required. We know that the minimum capacity must be at least the weight of the heaviest package and at most the sum of all package weights. We'll use these as our initial bounds for binary search.

In each iteration, we'll calculate the mid-value between our bounds and check if it's possible to ship all packages within the given days using this capacity. If it's possible, we try to find a smaller capacity by adjusting our upper bound. If it's not possible, we increase our lower bound. This approach ensures we find the smallest capacity that still allows all packages to be shipped within the given timeframe.

Step-by-step Algorithm

1. **Initialize Bounds:** Set the lower bound (`left`) to the maximum weight in the array and the upper bound (`right`) to the sum of all weights.

2. **Binary Search:**

   - While `left` is less than `right`, do the following:
     - Find the midpoint (`mid`) between `left` and `right`. This represents the current guess for the shipping capacity.
     - Initialize two variables, `need` (starting at 1, representing the number of days needed) and `cur` (current weight for the day, starting at 0).

3. **Check Feasibility:**

   - Iterate over the weights array. For each weight `w`:
     - If `cur + w` is greater than `mid`, increment `need` by 1 (indicating an additional day is needed) and reset `cur` to 0.
     - Add `w` to `cur`.

   - After iterating through all weights, check if `need` is greater than `D`. If so, it means the current capacity `mid` is too low, and we need to increase `left` to `mid + 1`. Otherwise, set `right` to `mid`.

4. **Find Minimum Capacity:** The loop continues until `left` meets `right`. The value of `left` at the end of the binary search is the minimum capacity required to ship all packages within `D` days.

Algorithm Walkthrough

Let's consider the input: `weights = [4, 8, 5, 2, 6, 3, 7, 9]`, days = `4`

1. **Initialization:**

   - `left = 9`, `right = 44`.

2. **First Iteration:**

   - `mid = (9 + 44)/2 = 26`.
   - Day 1: Ship 4, 8, 5, 2, 6 (total 25) - under capacity.
   - Day 2: Ship 3, 7 (total 10) - under capacity.
   - Day 3: Ship 9 (total 9) - under capacity.
   - Ships in 3 days, feasible. Set `right` to 26.

3. **Second Iteration:**

   - `mid = (9 + 26)/ 2 = 17`.
   - Day 1: Ship 4, 8, 5 (total 17) - under capacity.
   - Day 2: Ship 2, 6, 3 (total 11) - under capacity.
   - Day 3: Ship 7 (total 7) - under capacity.
   - Day 4: Ship 9 (total 9) - under capacity.
   - Ships in 4 days, feasible. Set `right` to 17.

4. **Third Iteration:**

   - `mid = (9 + 17)/2 = 13`.
   - Day 1: Ship 4, 8 (total 12) - under capacity.
   - Day 2: Ship 5, 2, 6 (total 13) - under capacity.
   - Day 3: Ship 3, 7 (total 10) - under capacity.
   - Day 4: Ship 9 (total 9) - under capacity.
   - Ships in 4 days, feasible. Set `right` to 13.

5. **Fourth Iteration:**

   - `mid = (9 + 13)/2 = 11`.
   - Day 1: Ship 4 (total 4) - under capacity.
   - Day 2: Ship 8 (total 8) - under capacity.
   - Day 3: Ship 5, 2 (total 7) - under capacity.
   - Day 4: Ship 6, 3 (total 9) - under capacity.
   - Day 5: Ship 7 (total 7) - under capacity.
   - Day 6: Ship 9 (total 9) - under capacity.
   - It ships in 6 days, which is not feasible. Set `left` to 12.

6. **Fifth Iteration:**

   - `mid = (12 + 13)/2 = 12`.
   - Day 1: Ship 4, 8 (total 12) - under capacity.
   - Day 2: Ship 5, 2 (total 7) - under capacity.
   - Day 4: Ship 6, 3 (total 9) - under capacity.
   - Day 5: Ship 7 (total 7) - under capacity.
   - Day 6: Ship 9 (total 9) - under capacity.
   - It ships in 5 days, which is not feasible. Set `left` to 13.

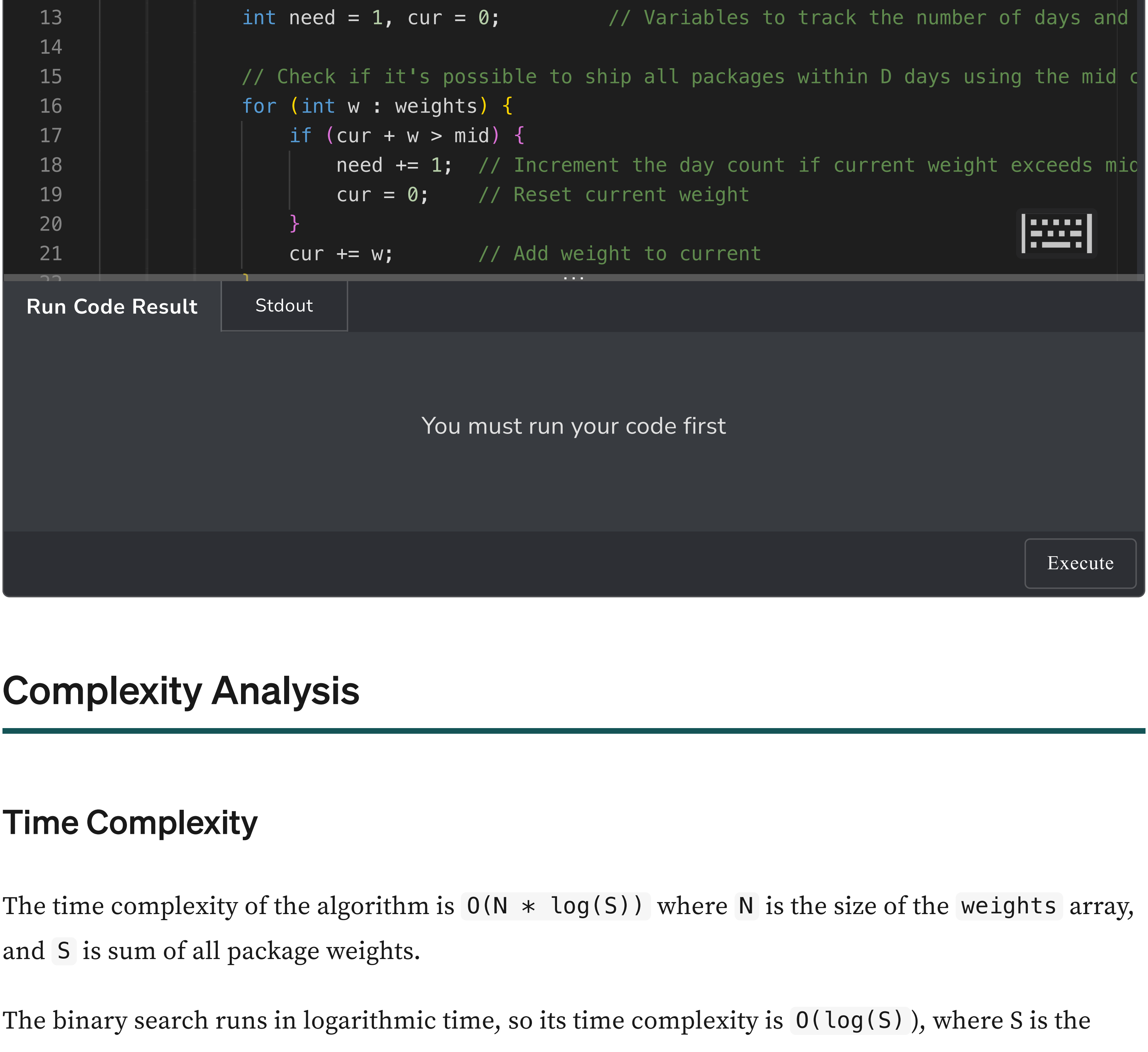
7. **Continue Iterations:**

   - `left` >= `right`, break the loop.

8. **Final Output:**

   - The loop ends with `left = 13`, which is the minimum capacity needed.

Code



```java
public class Solution {
    public int shipWithinDays(int[] weights, int D) {
        // Initialize the lower and upper bounds
        int left = 0, right = 0;
        for (int w : weights) {
            left = Math.max(left, w);  // Lower bound is the max weight
            right += w;                // Upper bound is the sum of all weights
        }

        // Binary search to find the minimum capacity
        while (left < right) {
            int mid = (left + right) / 2;  // Calculate the middle value
            int need = 1, cur = 0;         // Variables to track the number of days and

            // Check if it's possible to ship all packages within D days using the mid c
            for (int w : weights) {
                if (cur + w > mid) {
                    need += 1;  // Increment the day count if current weight exceeds mic
                    cur = 0;    // Reset current weight
                }
                cur += w;       // Add weight to current
```

You must run your code first

Complexity Analysis

Time Complexity

The time complexity of the algorithm is `O(N * log(S))` where `N` is the size of the `weights` array, and `S` is sum of all package weights.

The binary search runs in logarithmic time, so its time complexity is `O(log(S))`, where S is the sum of weights. For each iteration of the binary search, the algorithm iterates through the array once, which takes `O(N)` time. Thus, the overall time complexity is `O(N * log(S))`.

Space Complexity

The algorithm uses only a constant amount of extra space for variables, irrespective of the input size. The space used for input (weights array) is not considered in space complexity analysis. Thus, the space complexity is constant, `O(1)`.

← Previous
Capacity To Ship Packages Within D Days (Medium)

Next →
Count And Say (Medium)

☐ Mark as Completed